

KONTROL İŞLEMLERİ

Ardışık İşlem ve Kontrol İşlemleri

Nesne yönelimli programlama yöntemler, yapısal programlamadaki fonksiyonlar gibi kodlanır. Yani ilk önce işlenecek verileri taşıyan veri yapıları tanımlanır ve ardından bu verileri işleyen kontrol yapılarına ilişkin talimatlar yazılır. Şu ana karar örneğini verdiğimiz kodlarda veri yapısı olarak sadece değişkenler kullanılmıştır. Sonrasında ise giriş çıkış işlemleri talimatlar içeren programlar yazılmıştır. Programın icrası ilk talimattan başlar ve sırasıyla program bitene kadar devam eder. İşte programın icrasını değiştirmeyen bu tür talimatlara **ardışık işlem** (**sequential operation**) adı verilir. Ardışık işlemler aşağıdaki üç tür **talimattan** (**statement**) oluşur.

1. Değişkenlerin kimliklendirildiği değişken **tanımlamaları** (**identifier definition**):

```
int yariCap=3;
const float PI=3.14f;
float daireninAlani,daireninCevresi;
```

2. **İfadeler** (**expression**); sabit, değişken ve **işleç** (**operator**) içeren talimatlardır:

```
daireninAlani=PI*yariCap*yariCap;
float daireninCevresi=2.0f*PI*yariCap;
```

3. Klavyeden veri okuma ve konsola veri yazma gibi **giriş çıkış işlemleri** (**input output operation**):

```
Console.WriteLine("Karekökü Alınacak Sayıyı Giriniz");
string okunan = Console.ReadLine();
```

Kontrol işlemleri (**control operation**) ise programın icra sırasını değiştiren kontrol yapıları olup bu **talimatlar** (**statement**) olup üç türü vardır;

1. **Duruma göre seçimler** (**conditional choice**): **if**, **if-else** ve **switch** talimatları.
2. **İlişkisel döngü** (**relational loop**): **while**, **do-while** ve **for** talimatları.
3. **Dallanmalar** (**jump**): **continue**, **break**, **goto** ve **return** talimatları.

Kontrol işlemlerinde; talimat olarak yazılmış **mantıksal satırlar** (**logical sequence**) ile **fiziksel olarak icra edilen satırlar** (**physical sequence**) birbirinden farklıdır.

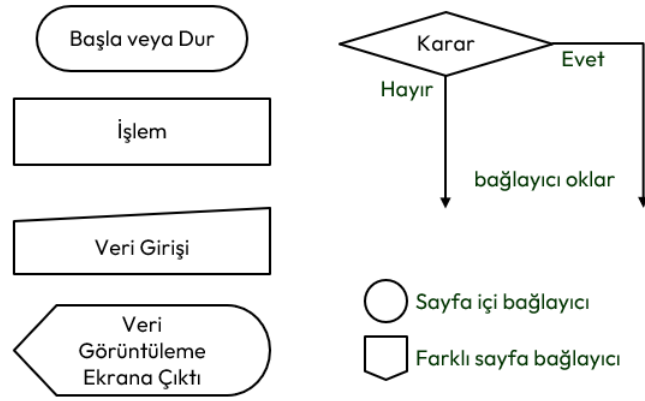
Akış Diyagramları

Kontrol işlemi içeren programın icrasını anlamak için akış diyagramlarını da anlamak gerekir. Akış diyagramı, adımların grafiksel gösterimleridir. Algoritmaları ve programlama mantığını temsil etmek için bir araç olarak bilgisayar biliminden ortaya çıkmıştır. Ancak diğer tüm işlem türlerinde kullanılmak üzere genişletilmiştir.

Günümüzde akış diyagramları, bilgileri göstermede ve akıl yürütmeye yardımcı olmada son derece önemli bir rol oynamaktadır. Karmaşık süreçleri görselleştirmemize veya sorunların ve görevlerin yapısını açık hale getirmemize yardımcı olurlar. Bir akış diyagramı, uygulanacak bir süreci veya projeyi tanımlamak için de kullanılabilir. Akış diyagramı çizilirken aşağıdaki kurallara uyulur;

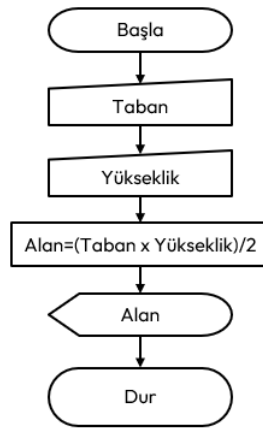
- Akış şemalarında tek bir başlangıç simgesi olmalıdır
- Bitiş simgesi birden çok olabilir.
- Karar simgesinin haricindeki simgelere her zaman tek giriş ve tek çıkış yolu bulunur.
- Bağlaç simgesi sayfanın dolmasından ötürü parçalanan akış şemasının öğelerini birleştirmede kullanılır.
- Simgeler birbirleri ile tek yönlü okla bağlanırlar.
- Okların yönü algoritmanın mantıksal işlem akışını tanımlar.

Aşağıda akış diyagramlarında kullanılan genel şekiller verilmiştir;



Şekil 9. Akış Diyagramları Genel Şekilleri

Aşağıda taban ve yüksekliği klavyeden girilen bir üçgenin alanını hesaplamak için bir akış diyagramı örneği verilmiştir.



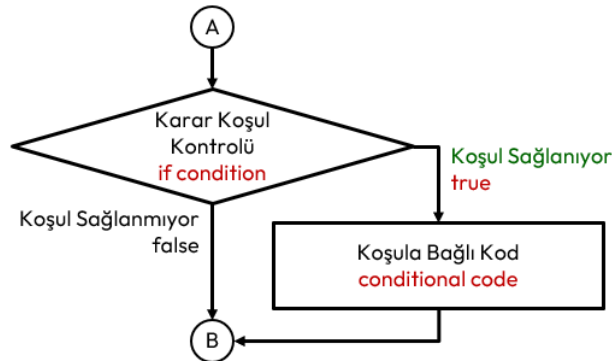
Şekil 10. Örnek Akış Diyagramı

Duruma Göre Seçimler

Duruma göre seçimler (conditional choice) programın akışını bir koşula göre değiştiren talimatlardır.

If Talimatı

If talimatı, karar vermeye (decision-making) ilişkin bir talimat olup bir koşula bağlı olarak programın icrasını değiştirir. Koşul (condition) ifadesi **true** ise koşul kodu (conditional code) icra edilir değilse icra edilmez. Koşul ifadesi (expression) test edilir ve sıfırdan farklı ise **true**, aksi halde **false** kabul edilir.



Şekil 11. If Talimatı İcra Akışı

Sözde kodu aşağıda verilmiştir.

```
if (koşul)
    koşul-kodu;
```

Programın icrası, ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir.

		yas	cinsiyet
1	int yas = 18;	18	?
2	char cinsiyet = 'K';	18	'K'
3	if (yas < 30)	18	'K'
4	Console.WriteLine("Genç");	18	'K'
5	if (cinsiyet == 'E')	18	'K'
	Console.WriteLine("Erkek");	18	'K'

Tablo 18. If Talimatı İçeren Bir Programın İcra Sırası

Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **yas** değişkenine **18** değeri atanır.
2. **cinsiyet** değişkenine **'K'** atanır.
3. **if** talimatındaki **koşul (condition)** test edilir. **yas<30** yani **18<30** testinde küçüktür işleci **true** verir. Bu nedenle izleyen **koşul kodu (conditional code)** icra edilir.
4. **Console.WriteLine("Genç");** koşul kodu olduğundan icra edilir.
5. **if** talimatındaki koşul test edilir. **cinsiyet=='E'** yani **'E'=='K'** testinde eşit mi işleci **false** verir. Dolayısıyla izleyen koşul kodu icra edilmez.

Bu durumda programın çalışması sonucu **Genç** çıktısı elde edilir. If talimatında **koşul kodu (conditional code)** her zaman **ardışık işlem (sequential operation)** olmaz. **if** gibi bir **kontrol işlemi (control operation)** de olabilir. Aşağıda **kademeli (compound/cascade)** if kullanımına ilişkin kod örneğinde ikinci **if**, birinci **if** talimatının koşul kodudur.

```
if (yas<30)
    if (yas<7)
        Console.WriteLine("Çocuk ");
```

Koşul kodu (conditional code) birden fazla talimattan oluşacak ise kod bloğu **{ }** içine alınır. Aşağıda verilen kod örneğinde ilk **if** talimatında **yas<30** ile test edilen koşul doğrulandığında kod bloğunun içindeki tüm talimatlar icra edilir.

```
if (yas<30) {
    if (yas<7)
        Console.WriteLine("Çocuk");
    if (yas<18)
        Console.WriteLine("Genç");
    if (yas>60)
        Console.WriteLine("Yaşlı");
}
```

Bahsedilen iki durum iç içe de olabilir. Buna ilişkin kod örneği de aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
}
if (cinsiyet=='K') {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```

If talimatında **koşul** (condition) her zaman tek bir test ifadesinden oluşmayabilir. Bahsedilen duruma ilişkin kod örneği de aşağıda verilmiştir.

```
if ((cinsiyet=='E') && (yas<18))
    Console.WriteLine("Genç Delikanlı");
if ((cinsiyet=='K') && (yas>60))
    Console.WriteLine("Yaşlı Kadın");
// if (10>rakam>5) // HATA!. Böyle bir şart yazılamaz.
Console.WriteLine("rakam 10 ile 5 arasında ");
// if (rakam>10>rakam>5) // HATA!. Böyle bir şart yazılamaz.
Console.WriteLine("rakam 10 ile 5 arasında ");
```

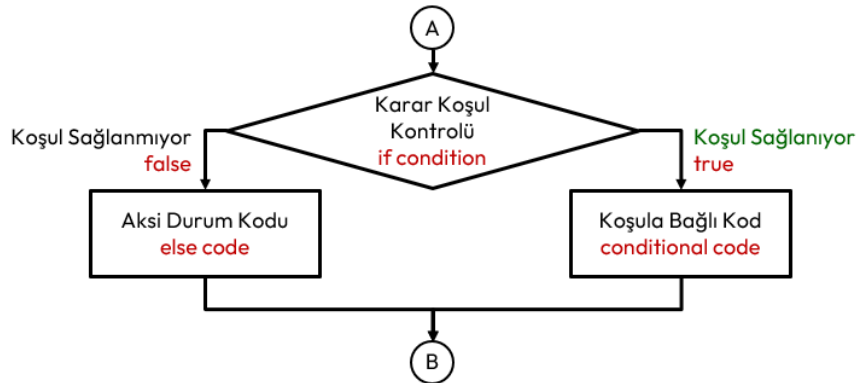
Bazen programcı tarafından aşağıdaki gibi **if** talimatları yazabilir.

```
if (true)
    Console.WriteLine("Bu metin konsola her zaman yazılır.");
if (false)
    Console.WriteLine("Bu metin konsola hiçbir zaman yazılmaz!");
```

If-Else Talimatı

If-Else Talimatı, bir başka **karar verme** (decision-making) talimatı olup bir koşula bağlı olarak programın icrasını değiştirir. If talimatına benzer şekilde **koşul** (condition) ifadesi **true** ise **koşul kodu** (conditional code) icra edilir, değilse **aksi durum kodu** (else code) icra edilir.

```
if (koşul)
    koşul-kodu;
else
    aksi-durum-kodu;
```



Şekil 12. If-Else Talimatı Sözde Kodu ve İcra Akışı

Programın icrası, ilk talimatla başlar ve solda verilen sırayla icra edilir. Sağda ise her icra sonrası değişkenlerin değerleri gösterilmiştir

		yas	cinsiyet
1	int yas = 65;	65	?
2	char cinsiyet = 'K';	65	'K'
3	if (yas < 50)	65	'K'
	Console.WriteLine("Genç ");	65	'K'
4	else	65	'K'
5	Console.WriteLine("Yaşlı ");	65	'K'

Tablo 19. If-Else Talimatı İçeren Bir Programın İcra Sırası

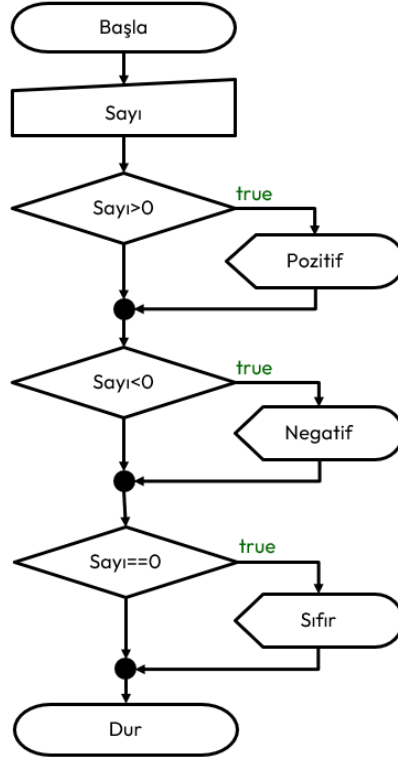
Her talimatın icrasında neler olduğu aşağıda verilmiştir;

1. **yas** değişkenine **65** değeri atanır.
2. **cinsiyet** değişkenine **'K'** atanır.
3. if talimatındaki **koşul** (condition) test edilir. **yas<50** yani **65<50** testinde küçüktür işleci **false** verir. Test sonucu **false** olduğundan aksi **durum kodu** (else code) icra edilir.
4. Programın icrası **else** ifadesinden devam eder.
5. **Console.WriteLine("Yaşlı ");** aksi durum kodu olduğundan icra edilir.

Bu durumda programın çalışması sonucu **Yaşlı** çıktısı elde edilir. If talimatında olduğu gibi bu talimatta da **koşul kodu** (conditional code) ya da **aksi durum kodu** (else code) birden fazla talimattan oluşacak ise kod bloğu { } içine alınır. Buna ilişkin kod örneği aşağıda verilmiştir.

```
if (cinsiyet=='E') {
    if (yas<7)
        Console.WriteLine("Erkek Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Delikanlı");
    if (yas>60)
        Console.WriteLine("Yaşlı Adam");
} else {
    if (yas<3)
        Console.WriteLine("Kız Bebek");
    if (yas<7)
        Console.WriteLine("Kız Çocuk");
    if (yas<18)
        Console.WriteLine("Genç Kız");
    if (yas>60)
        Console.WriteLine("Yaşlı Kadın");
}
```

Örnek olarak klavyeden girilen bir sayının sıfırdan küçük mü? Büyük mü? ya da sıfıra eşit mi? Olduğunu bulan programı yazalım. Programın icra akışı aşağıdaki şekilde olacaktır.



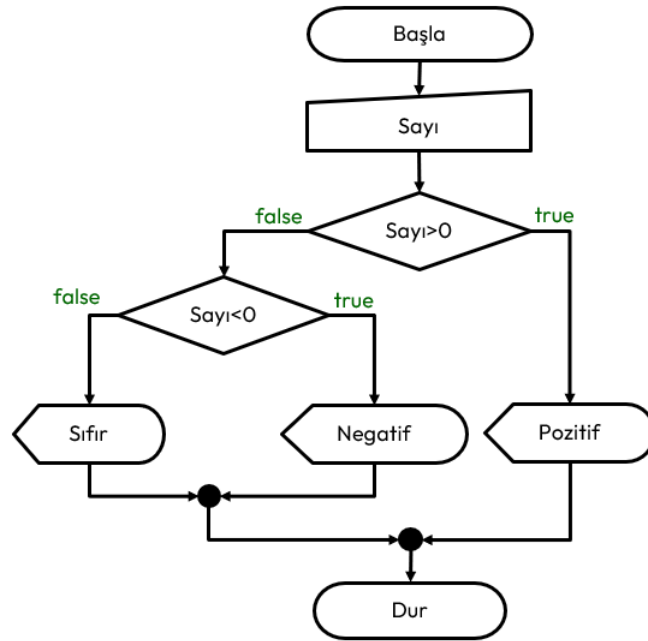
Şekil 13. Programın If Talimatlarıyla Yazılması Halinde İcra Akışı

Bu icra akışını sağlayan program aşağıda verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi > 0) // sayının pozitif olup olmadığı test ediliyor.
    Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
if (sayi < 0) // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
if (sayi == 0) // sayının sıfır olup olmadığı test ediliyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
```

Bu akışta ilk if talimatında yapılan test doğru çıkarsa geri kalan testlerin yapılmasına gerek kalmaz. Benzer şekilde ilk iki test geçilirse üçüncü if talimatındaki teste gerek yoktur. Bu durumda aynı program if-else talimatlarıyla aşağıdaki şekilde yeniden yazabiliriz.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Sayi Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
if (sayi > 0)
{ // sayının pozitif olup olmadığı test ediliyor.
  Console.WriteLine("Pozitif"); // Burada sayının pozitif olduğu biliniyor.
}
else
{ // Burada sayının pozitif olmadığı biliniyor.
  if (sayi < 0)
  { // sayının negatif olup olmadığı test ediliyor.
    Console.WriteLine("Negatif"); // Burada sayının negatif olduğu biliniyor.
  }
  else
  { // Burada sayının pozitif ve negatif olmadığı test biliniyor.
    Console.WriteLine("Sıfır"); // Burada sayının sıfır olduğu biliniyor.
  }
}
}
```



Şekil 14. Programın If-else Talimatlarıyla Yazılması Halinde İcra Akışı

Program incelendiğinde sayının pozitif girilmesi halinde sadece ilk if talimatındaki test geçecek ve else sonrası aksi durum koduna ilişkin blok icra edilmeyecektir. Sayının negatif girilmesi halinde ilk if talimatının else bloğuna girilecek ve blok içindeki ilk if talimatındaki test geçecek ve bu if talimatının else kısmı çalıştırılmayacaktır. Sayı sıfır girilmiş ise blok içindeki else kodu çalıştırılacaktır. Bunların dışında blok içinde tek bir if-else talimatı bulunmaktadır. Dolayısıyla blok içine almaya gerek de yoktur. Bu durumda kodun nihai hali aşağıda verilmiştir.

```
if (sayi>0)
    Console.WriteLine("Pozitif");
else if (sayi<0)
    Console.WriteLine("Negatif");
else
    Console.WriteLine("Sıfır");
```

Sarkan Else

Aşağıdaki program örneği incelendiğinde else, hangi if talimatına aittir? Böyle bir kod programcı tarafından yazılabilir ve derleyici buna hiçbir sıkıntı çıkarmaz.

```
if (cinsiyet=='E') if (yas<18) Console.WriteLine("Delikanlı"); else Console.WriteLine("Adam");
```

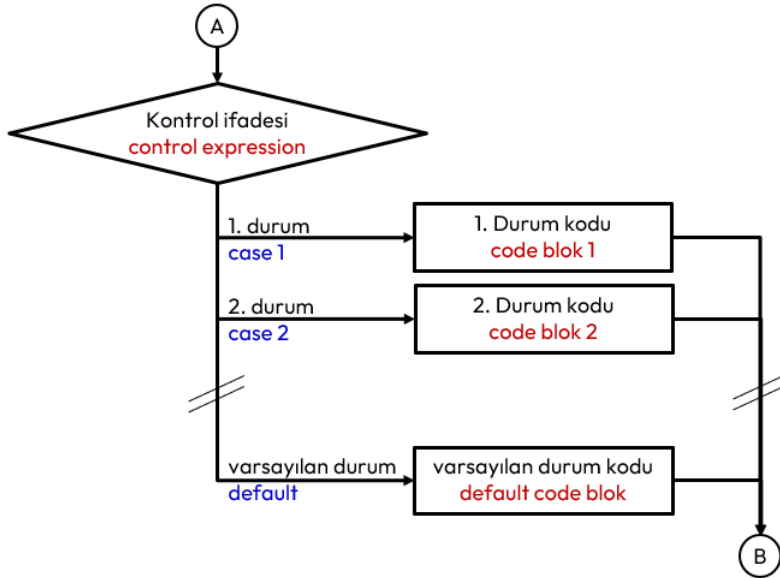
Bu durumda kodu aşağıdaki gibi okunaklı hale getirdiğimizde ilk **if** talimatının **koşul kodunun** (**conditional code**) ikinci if-else talimatı olduğu görülmektedir. Bu problem **sarkan else** (**dangling else**) problemi olarak bilinir. Kod bloğu kullanılmadan yazılan bu tip kodlarda else her zaman kendinden bir önceki if talimatına aittir.

```
if (cinsiyet=='E')
    if (yas<18)
        Console.WriteLine("Delikanlı");
    else
        Console.WriteLine("Adam");
```

Switch Talimatı

Switch talimatı, bir kontrol **ifadesi** (**expression**) sonucunda birden fazla alternatif arasında seçim yapılmasını sağlar. Sözde kodu aşağıda verilmiştir;

```
switch (kontrolifadesi) {
    case alternatif-1:
        alternatif-1-kodu;
        break;
    case alternatif-2:
        alternatif-2-kodu;
        break;
    // ...
    default:
        varsayılan-alternatif-kodu;
}
```



Şekil 15. Switch Talimatı Sözde Kodu ve İcra Akışı

Switch talimatına ilişkin kurallar;

1. **Kontrol ifadesi** (**control expression**) bir kez değerlendirilir.
2. Kod bloğu zorunludur. Yani bloğu olmayan bir **switch** talimatı yazılamaz!
3. Diğer birçok dilin aksine her bir alternatif case ifadesi, kontrol ifadesinin sonucunu bir **desen** (**pattern**) ile seçer ve iki nokta ile biter. Desenler, **ilişkisel desen** (**relational pattern**) ve **sabit desen** (**constant pattern**) olmak üzere iki türüdür.

4. Blok için yazılan kod sonuna **break;** talimatı yazılarak **switch** bloğu dışına çıkılır. Zorunlu değildir, yazılmaz ise bir sonraki durum için yazılan kod icra edilir.
5. Alternatiflerin tümü yazılmayabilir.

Blok sonunda yer alacak şekilde üzerinde yer alan alternatifler dışında kalan tüm alternatifler için **default:** etiketli alternatif yazılabilir. Zorunlu değildir.

Aşağıda klavyeden girilen bir sayının 4 rakamına bölünmesinde kalanı gösteren bir program verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayi;
Console.WriteLine("Bir Sayı Giriniz:");
sayi=Convert.ToInt32(Console.ReadLine());
switch (sayi % 4)
{ //konrol ifadesi bir işlem içerir
    case 3:
        Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 3 dir.");
        break;
    case 2:
    case 1:
        Console.WriteLine("Sayı 4 rakamına bölündüğünde kalan 2 veya 1 dir.");
        break;
    default: // Başka alternatif yoktur.
        Console.WriteLine("Sayı 4 Rakamına TAM Bölünür");
        break;
}
```

C ve C++ dillerinde case ifadelerinde sadece tamsayı değişmezler kullanılır. C# dilinde ise desenler kullanılabilir. Aşağıda gerçek sayı olarak yapılan bir ölçüm için alternatiflerin desen ile seçimi için yazılmış bir program örneği verilmiştir.

```
// See https://aka.ms/new-console-template for more information
double ölçüm;
Console.WriteLine("Ölçüm Giriniz:");
ölçüm = Convert.ToDouble(Console.ReadLine());
switch (ölçüm)
{
    case < 0.0:
        Console.WriteLine($"Ölçülen değer {ölçüm}, çok düşük.");
        break;
    case > 15.0:
        Console.WriteLine($"Ölçülen değer {ölçüm}, çok yüksek.");
        break;
    case double.NaN:
        Console.WriteLine("Yanlış Ölçüm.");
        break;
    default:
        Console.WriteLine($"Ölçülen değer: {ölçüm}.");
        break;
}
```

Üçlü Şart İşleci

Daha önce *Temel* İşleçler başlığı altında işlenenlere ek olarak, **ardışık işlemlerde** (**sequential operation**) if talimatlarına gerek kalmadan bir koşula göre **ifadeler** (**expression**) işlenecek ise **üçlü şart işleci** (**conditional tenary operator**) kullanılır. Aşağıda üçlü işlecin iki sözdizimi verilmiştir;

```
değişken = koşul ? kosulifadesi : aksidurumifadesi;
```

Aşağıda oy kullanma durumu bu işleçle işlenmiştir;

```
// See https://aka.ms/new-console-template for more information
int yas;
```



```
Console.WriteLine("Yaşınız?: ");
yas=Convert.ToInt32(Console.ReadLine());
string metin=(yas >= 18) ? "Oy Kullanabilirsin.": "Oy Kullanamazsın!";
Console.WriteLine(metin);
```

Aşağıda başka kullanımlarına ilişkin kod örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
int sayı;
bool tek, çift;
Console.WriteLine("Bir Sayı Giriniz: ");
sayı=Convert.ToInt32(Console.ReadLine());
tek = (sayı % 2==0) ? true : false;
çift =!tek;
int sonuç1 = tek ? sayı + 30 : sayı + 40;
int sonuç2 = çift ? sayı * 30 : sayı * 40;
Console.WriteLine($"Sonuç1: {sonuç1}, Sonuç2: {sonuç2}");
```

null Kaynaştırma İşleci

`null` **kaynaştırma işleci** (**null coalescing operator**) ile sol taraftaki işlenen `null` ise `null` olabilir bir tip için varsayılan bir değer belirtmenize olanak tanır.

```
string testString = null;
Console.WriteLine("Belirtilen Metin - " + (testString ?? "null'dür."));
```

Mantıksal olarak aşağıdaki koda eşdeğerdir;

```
string testString = null;
if (testString == null)
{
    Console.WriteLine("Belirtilen Metin - null'dür.");
}
else
{
    Console.WriteLine("Belirtilen Metin - " + testString);
}
```

`??=` işleci (**null coalescing assignment operator**) yalnızca değişken `null` ise bir değer atar;

```
int? number = null;
int result = number ?? 100; // 'number' null ise 100 atar
string message = null;
message ??= "Default Message"; // 'message' null ise atama yapar
```

İlişkisel Döngüler

İşlemciler iyi bir sayıcıdır. CPU içindeki **kaydediciler** (**registers**) sayma işlevini birçok matematiksel işlemi yapmak için de kullanılır. Program akışında belli problemlerin çözümünde, gerçekleştirilen belli adımların tekrarlanması sonucu gidilir. Bu tip problemler şimdiye kadar olan yöntemlerle yapılırsa, tekrar tekrar aynı kodu yazmak zorunda kalırız. Yazdığımız kodları tekrar tekrar icra edebilmek için **ilişkisel döngü** (**relational loop**) talimatlarını kullanırız.

Konsola 10 defa ismimizi yazdıran bir program örneğini ele alalım. Burada konsolda **satır başı** (**new line**) yapmak için konsola **Environment.NewLine** gönderilir.

```
// See https://aka.ms/new-console-template for more information
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

```
Console.WriteLine("İLHAN");
Console.WriteLine("İLHAN");
```

Program incelendiğinde aynı `Console.WriteLine` talimatının tekrar tekrar yazıldığını görürüz. Bu istenmeyen bir durumdur. Bunu yapmamak için sayaç olarak kullanılan bir değişken kullanırız.

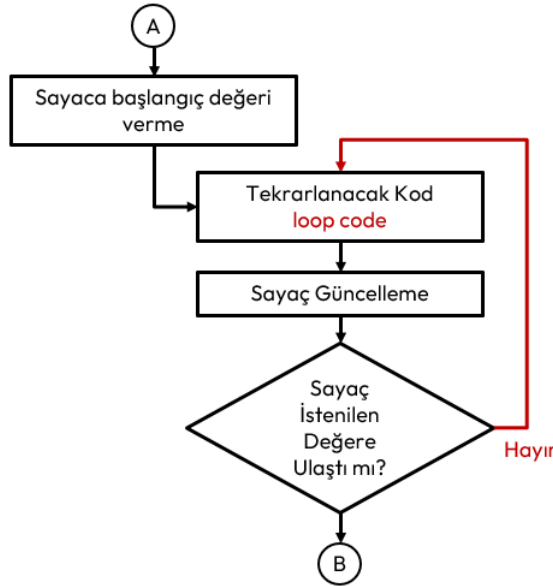
Sayaç Kontrollü Döngüler

Belirlenen bir **sayaç** (**counter**) değişkeninin istenilen bir değere ulaşip ulaşmadığı kontrol ederek kodun tekrar tekrar icra edilmesi sağlanır.

Yapısal programlama öncesinde bu döngü **goto** talimatıyla sağlanır. İcra sırasını etiketlenen bir yer olarak değiştirmek için **goto** talimatı kullanılır. Etiketlere kimlik verilirken yine **değişken kimliklendirme** (**identifier definition**) kuralları uygulanır.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0;
Etiket:
Console.WriteLine("İLHAN");
sayaç++;
if(sayaç<10) goto Etiket;
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu üç adet talimat içeren **mantıksal satır** (**logical sequence**) olmasına karşın 30 adet **fiziksel satır** (**physical sequence**) icra edilmiştir.



Şekil 16. Sayaç Kontrollü Döngü İcra Akışı

C programlama dilinde ara seviye bir dil olduğundan bu talimat desteklenir. 1968 Yılında *Edsger W. Dijkstra* GOTO/JUMP TO ifadelerini zararlı olarak ilan edilmiştir¹⁷. GOTO kullanmamak için; while, do-while ve for **kontrol talimatları** yapısal programlamaya eklenmiştir.

While Talimatı

While döngüsü, **koşul** (**condition**) **true** olduğu sürece **tekrarlanacak kodu** (**loop code**) icra eder.

Tekrarlanacak kod, tek bir **talimat** (**statement**) olabileceği gibi bir kod bloğu da olabilir. Sözde kodu aşağıda verilmiştir;

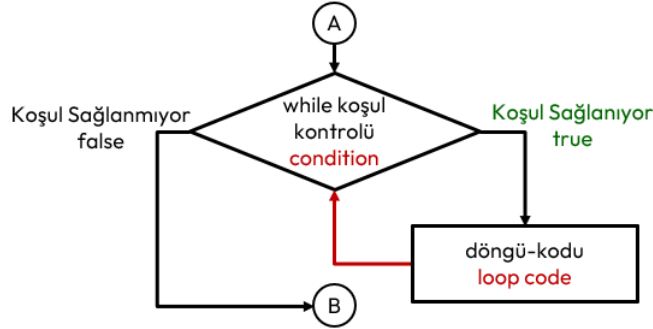
```
while (koşul)
    döngü-kodu;
```

Sayaç Kontrollü Döngüler başlığında verilen döngü, While talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
```

¹⁷ <https://homepages.cwi.nl/~storm/teaching/reader/Dijkstra68.pdf>

```
while (sayaç<10) // while koşul kontrolü
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
}
```



Şekil 17. While Talimatı İcra Akışı

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir. Her koşul kontrolü sonrası sayaç artırılacaktır.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça İlk Değer 0 Olarak Verildi
while (sayaç++<10) // while koşul kontrolü ve sayaç güncelleme
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
```

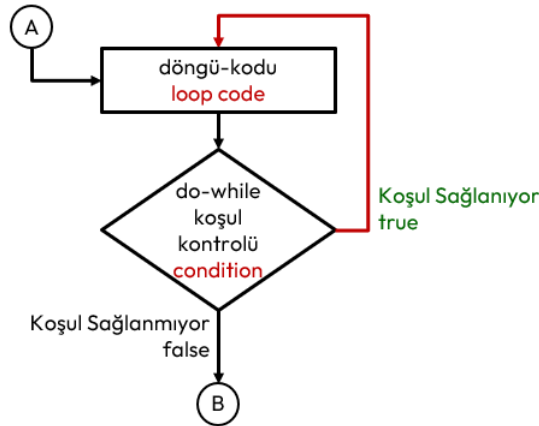
Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren mantıksal satır (logical sequence) içermesine rağmen 10 adet fiziksel satır (physical sequence) icra edilmiştir.

Do-While Talimatı

Do-While döngüsü, **do** ile **while** saklı kelimeleri arasındaki **tekrarlanacak kodu** (loop code) **koşul** (condition) **true** olduğu sürece tekrarlayarak icra eder. Tekrarlanacak kod tek bir talimat (statement) olabileceği gibi bir kod bloğu da olabilir. While döngüsünde koşul kontrolü en başta yapılır, bu döngüde ise döngü bloğu en az bir kez çalıştırdıktan sonra koşul kontrolü yapılır.

Sözde kodu aşağıda verilmiştir;

```
do
    döngü-kodu;
while (koşul);
```



Şekil 18. Do-While Talimatı ve İcra Akışı

Sayaç Kontrollü Döngüler başlığında verilen döngü, Do-While talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
```

```
{ // döngü bloğu
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
    sayaç=sayaç+1; //Sayaç Güncelleme
} while (sayaç<10); // Do-While koşul kontrolü
```

Sayaç güncelleme ifadesi koşul ifadesine eklenerek program daha da kısaltılabilir.

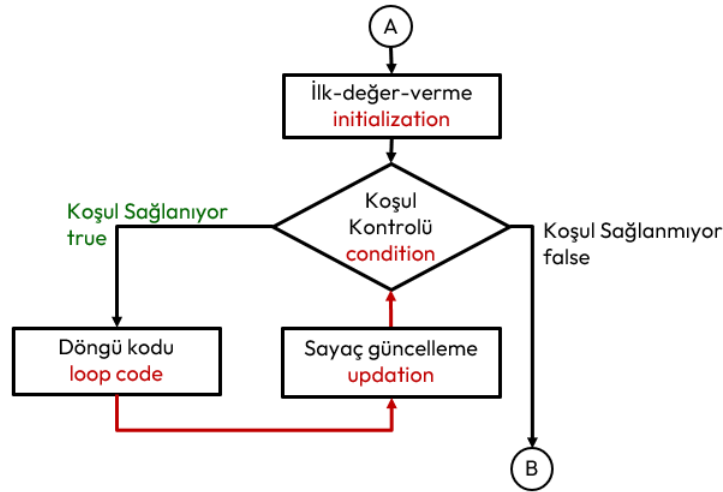
```
// See https://aka.ms/new-console-template for more information
int sayaç=0; //sayaça ilk Değer 0 Olarak Verildi
do
    Console.WriteLine("İLHAN"); // Döngüde çalıştırılacak kod.
while (sayaç++<10); // Do-While koşul kontrolü ve sayaç güncelleme
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu 1 adet talimat içeren **mantıksal satır** (logical sequence) içermesine rağmen 10 adet **fiziksel satır** (physical sequence) icra edilmiştir.

For Talimatı

For döngüsü özellikle tekrar edilen işlemlerin sayısı belli olduğunda kullanılan bir döngü yapısıdır. Sayaç kontrollü döngüler için en iyi seçimdir. Sözde kodu aşağıda verilmiştir.

```
for (ilk-değer-verme; koşul-kontrolü; sayaç-güncelleme)
    döngü-kodu;
```



Şekil 19. For Talimatı İcra Akışı

For talimatı aşağıdaki şekilde icra edilir;

1. İlk değer verme ifadesi bir kez icra edilir.
2. Sonra koşul kontrolü yapılır. Eğer test sonucu **true** ise döngü kodu icrasına geçilir. Aksi halde döngüden çıkılır.
3. Döngü kodu icra edilir.

Döngü kodu icrası bitince sayaç güncellemeleri yapılır ve tekrar ikinci adımdaki koşul kontrolüne dönülür. Sayaç Kontrollü Döngüler başlığında verilen döngü, For talimatı ile aşağıdaki şekilde yazılabilir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç=0; sayaç<10; sayaç++)
    Console.WriteLine("İLHAN");
```

Buradaki döngü mantıksal olarak iki satır talimattan oluşmaktadır. Ancak fiziksel olarak 10 satır icra edilmiştir. Aşağıda konsola yazdığımız her bir satırın başına satır numarası koyan bir program gösterilmektedir. Ayrıca sayaç sadece for bloğunda kullanılacak ise ilk değer verme ifadesinde tanımlanabilir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç=0; sayaç<10; sayaç++)
```

```
Console.WriteLine($"{sayaç}-İLHAN");
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu bir adet talimat içeren **mantıksal satır** (logical sequence) içermesine rağmen 10 adet **fiziksel satır** (physical sequence) icra edilmiştir.

For döngüsünde bir sayaç ile çalışılabileceği gibi birden fazla sayaç ile de çalışılabilir. Hem ilk değer verme hem de sayaç güncellemede birden fazla sayaca ilişkin işlem yapılabilir. Bunun için **virgül işleci** (comma operator) kullanılır.

```
// See https://aka.ms/new-console-template for more information
for (int artanSayaç=0, azalanSayaç=10;
     artanSayaç<10;
     artanSayaç++, azalanSayaç--)
    Console.WriteLine($"{artanSayaç}-İLHAN-{azalanSayaç}");
/* Program Çıktısı:
0-İLHAN-10
1-İLHAN-9
2-İLHAN-8
3-İLHAN-7
4-İLHAN-6
5-İLHAN-5
6-İLHAN-4
7-İLHAN-3
8-İLHAN-2
9-İLHAN-1
*/
```

Aşağıda klavyeden girilen iki sayı aralığında tek ve çift sayıların toplamını bulan ve ekrana yazan programı verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayaç, sayı1,sayı2;
int tekToplam=0,çiftToplam=0;
Console.WriteLine("Birinci Sayıyı Giriniz: ");
sayı1=Convert.ToInt32(Console.ReadLine());
Console.WriteLine("İkinci Sayıyı Giriniz: ");
sayı2=Convert.ToInt32(Console.ReadLine());
if (sayı2<sayı1) { //sayı2, sayı1 den büyük olmalı.
    int temp=sayı1; //küçük ise yer değiştiriyoruz.
    sayı1=sayı2;
    sayı2=temp;
}
for (sayaç=sayı1; sayaç<=sayı2; sayaç=sayaç+1) {
    if (sayaç%2==1) //sayaç tek mi?
        tekToplam+=sayaç;
    else
        çiftToplam+=sayaç;
}
Console.WriteLine($"Tek Toplam:{tekToplam} Çift Toplam:{çiftToplam}");
/* Program Çıktısı:
Birinci Sayıyı Giriniz:
11
İkinci Sayıyı Giriniz:
1
Tek Toplam:36 Çift Toplam:30
*/
```

İç İçe Döngü Kodu

Döngüler içinde yer alan **döngü kodu** (loop code) bir başka döngüyü içerebilir. Örneğin ekrana satır ve sütun içeren bir şekil oluşturmak istediğimizde iç içe döngü kullanırız. Aşağıda buna ilişkin bir örnek verilmiştir. İlk for döngüsünün tekrarlanan kodu ikinci for döngüsüdür.

```
// See https://aka.ms/new-console-template for more information
```

```
int satır,sütun;
for (satır=0; satır<5; satır++)
{
    for (sütun=0; sütun<4; sütun++)
        Console.Write("*");
    Console.WriteLine();
}
/* Program Çıktısı:
****
****
****
****
****
*/
```

Bir başka örnek olarak elemanları, satır ve sütun çarpımları olan 4x5 boyutlarındaki matrisi ekrana yazdıran program;

```
// See https://aka.ms/new-console-template for more information
int satır, sütun;
//BAŞLIK Yazdırılacak:
Console.Write("Sütunlar:");
for (sütun = 0; sütun < 4; sütun++)
    Console.Write($"{sütun,2} ");
Console.WriteLine();
//Matris Yazılacak:
for (satır = 0; satır < 5; satır++)
{
    Console.Write($"Satır {satır}:");
    for (sütun = 0; sütun < 4; sütun++)
        Console.Write("{0,2} ",satır*sütun);
    Console.WriteLine();
}/* Program Çıktısı:
Sütunlar: 0  1  2  3
Satır  0: 0  0  0  0
Satır  1: 0  1  2  3
Satır  2: 0  2  4  6
Satır  3: 0  3  6  9
Satır  4: 0  4  8 12
*/
```

Döngülerde Gözcü Kontrolü

Döngüler her zaman bir sayaca bağlı çalıştırılmaz. Bir döngüden çıkış koşulu döngü kodu içerisinde üretilen bir değere bağlı ise bu değere **gözcü değeri** (**sentinel value**) adı verilir. Buradaki gözcü değeri beklenen bir değer dışındaki bir değerdir.

Buna örnek olarak klavyeden 0 girilene kadar girilen rakamları toplayan bir program verilmiştir.

```
// See https://aka.ms/new-console-template for more information
int sayı; /* okunan tamsayı */
int toplam = 0; /* girilen sayıların toplamı. başlangıçta 0*/
do
{
    Console.WriteLine("Bir sayı giriniz (Bitirmek için 0): ");
    sayı=Convert.ToInt32(Console.ReadLine());
    toplam += sayı; // sayı 0 girilse bile toplam etkilenmez
} while (sayı != 0); //sentinel value 0
Console.WriteLine($"Girilen Sayıların Toplamı: {toplam}");
/*Program Çıktısı:
Bir sayı giriniz (Bitirmek için 0):
10
Bir sayı giriniz (Bitirmek için 0):
20
```

```

Bir sayı giriniz (Bitirmek için 0):
30
Bir sayı giriniz (Bitirmek için 0):
5
Bir sayı giriniz (Bitirmek için 0):
0
Girilen Sayıların Toplamı: 65
*/

```

Aşağıda pozitif sayı girilmesini zorlayan bir kod örneği verilmiştir.

```

// See https://aka.ms/new-console-template for more information
int sayı, pozitifsayı = 0;
do
{ /* Pozitif girilmesini zorluyoruz */
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı=Convert.ToInt32(Console.ReadLine());
    if (sayı <= 0)
        Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
} while (sayı <= 0);
pozitifsayı = sayı;
Console.WriteLine($"Girilen pozitif tamsayı {pozitifsayı} ");

```

Aynı örnek bir başka şekilde aşağıdaki gibi kodlanabilir.

```

// See https://aka.ms/new-console-template for more information
int sayı, pozitifsayı = 0;
Console.WriteLine("Lütfen pozitif sayı giriniz: ");
sayı = Convert.ToInt32(Console.ReadLine());
while (sayı <= 0)
{
    Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı = Convert.ToInt32(Console.ReadLine());
}
pozitifsayı = sayı;
Console.WriteLine($"Girilen pozitif tamsayı {pozitifsayı} ");

```

BigInteger Veri Tipi

Fibonacci dizisi, her sayının kendisinden önceki iki sayının toplamı olduğu bir dizidir. Bu basit algoritma, en az 1000 ondalık basamağa ulaşana kadar Fibonacci sayılarını yineler, sonra da yazdırır. Bu değer, bir **ulong** veri tipinin tutabileceğinden bile önemli ölçüde daha büyüktür. Teorik olarak, **BigInteger** sınıfındaki tek sınır, uygulamanızın tüketebileceği RAM miktarıdır;

```

// See https://aka.ms/new-console-template for more information
using System.Numerics;
BigInteger l1 = 1;
BigInteger l2 = 1;
BigInteger current = l1 + l2;
while (current.ToString().Length < 1000)
{
    l2 = l1;
    l1 = current;
    current = l1 + l2;
}
Console.WriteLine(current);

```

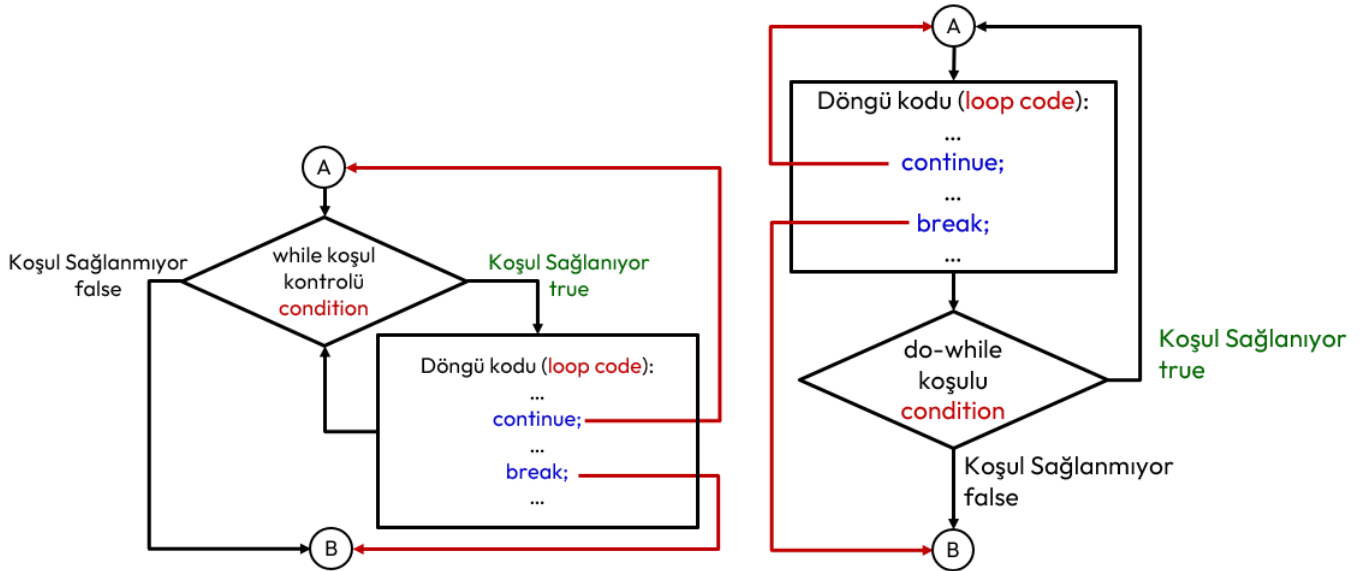
Dallanmalar

Dallanmalar (**jump**) programın akışını bir başka talimata yönlendiren talimatlardır.

Continue ve Break Talimatları

Break talimatı ile Switch talimatında ivedilikle switch bloğu dışına çıkıldığı anlatılmıştı. Benzer şekilde **do**, **while** ve **for** talimatlarında da ivedilikle yinelemeyi bitirip döngü bloğu dışına çıkılmasını sağlar. Continue talimatı yalnızca geçerli yinelemeyi (iteration) sonlandırır ve sonraki yinelemelerle devam eder. Dolayısıyla bu talimatlar sadece döngü kodları (loop code) içinde kullanılabilir.

break Talimatı döngüyü sonlandırır ve program icrası döngü sonrası ilk talimattan devam eder. Bu talimat, aynı şekilde While, Do-While ve For döngüleri ile kullanılabilir. **continue** Talimatı ise bir sonraki yinelemeyi yapmak için kullanılır. Bu talimatlarının döngü kodlarında kullanılması halinde icra akışı aşağıdaki şekilde verilmiştir.

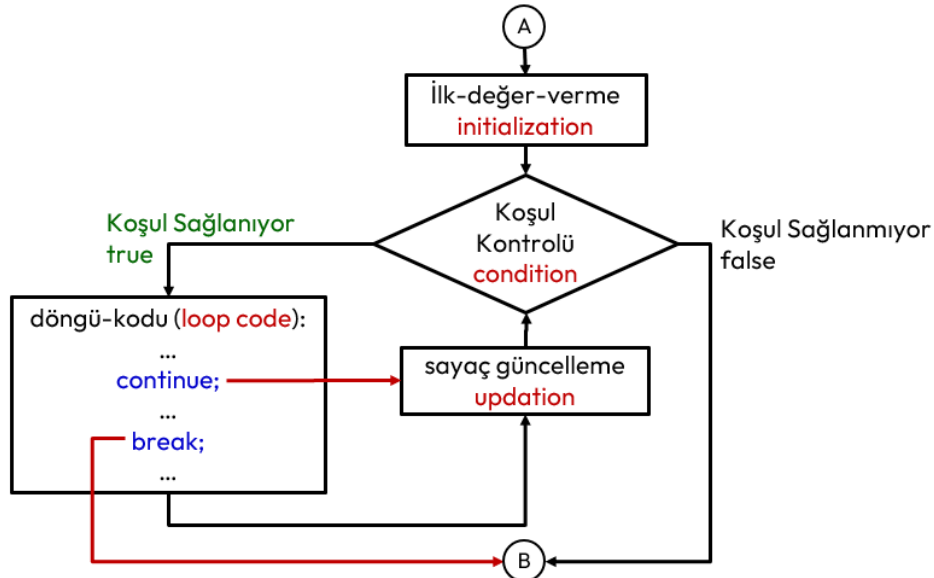


Şekil 20. While ve Do-While Döngü Kodunda break ve continue Talimatları İcra Akışı

While döngüsünde, döngü bloğunda **continue** kullanmadan önce koşulu değiştirecek bir talimat vermeliyiz. Aksi halde sonsuz döngüye girmiş oluruz. Do-While döngüsünde eğer **continue** talimatı bir şarta bağlanmalıdır. Yoksa sonsuz döngüye girilebilir.

Benzer durum for döngüsü için de geçerlidir. Ancak for döngüsünde **continue** talimatı sonrasında bir sonraki yineleme için önce sayaç güncelleme ifadeleri icra edilir ve sonra koşul testi yapılır. Bu durum göz önüne alınarak bu talimat kullanılmalıdır.

Bu talimatlar döngü kodu içinde amacımıza uygun olarak birden çok kullanabiliriz.



Şekil 21. For Döngü Kodunda break ve continue Talimatları İcra Akışı

Aşağıda 0 ile 15 arasındaki sayıların beşe bölünenleri ve 10 dışındaki sayılar ekrana yazdıran bir program örneği verilmiştir.

```
// See https://aka.ms/new-console-template for more information
for (int sayaç = 0; sayaç <= 15; sayaç = sayaç + 1)
{
    if (sayaç < 7) //sayaç'ın değeri 7 den küçük ise
        continue; //bir sonraki yinelemeye geç
    if (sayaç == 10) //sayaç değeri 10 ise
        continue; //bir sonraki yinelemeye geç
    if (sayaç % 5 == 0) // ssayaç değeri 5 e bölünüyorsa
        continue; //bir sonraki yinelemeye geç
    Console.WriteLine($"{sayaç}");
}
/*Program Çıktısı:
7
8
9
11
12
13
14
*/
```

Yukarıdaki örnek incelendiğinde işaretli döngü bloğu üç adet talimat içeren mantıksal satır (logical sequence) olmasına karşın 42 adet fiziksel satır (physical sequence) icra edilmiştir. Bazı continue talimatları Console.WriteLine talimatının icrasını engellemiştir.

Aşağıda pozitif sayı girilmesini zorlayan sonsuz döngü içeren program verilmiştir. Programda döngü, pozitif sayı girildiğinde break talimatı ile kırılmaktadır.

```
// See https://aka.ms/new-console-template for more information
int sayı;
int pozitifsayı = 0;
while (true) /* Sonsuz döngü */
{
    Console.WriteLine("Lütfen pozitif sayı giriniz: ");
    sayı=Convert.ToInt32(Console.ReadLine());
    if (sayı <= 0)
        Console.WriteLine("HATA:Pozitif Sayı GİRMEDİNİZ!");
    else
        break; //döngüden çık
}
pozitifsayı = sayı;
Console.WriteLine("Girilen pozitif tamsayı: {pozitifsayı} ");
```

Return Talimatı

Yöntemlerde çokça kullanacağımız return talimatı yapısal programlamadaki bir fonksiyonun gibidir. Yöntemler de fonksiyonlar gibi kodlanır. Üreteceği değerleri return talimatı ile geri döndürür.

```
namespace ConsoleApp1
{
    internal class Program
    {
        static int KonsoldanBirTamsayıOku()
        {
            Console.WriteLine("Bir Tamsayı Giriniz:");
            int okunan = Convert.ToInt32(Console.ReadLine());
            return okunan;
        }
        static double KonsoldanBirGerçekSayıOku()
        {
            Console.WriteLine("Bir Gerçek Sayı Giriniz:");
```

```
        double okunan = Convert.ToDouble(Console.ReadLine());
        return okunan;
    }
    static void TamsayıyıYaz(int tamsayı)
    {
        Console.WriteLine($"{tamsayı}");
    }
    static int İşaretiniDeğiştir(int tamsayı)
    {
        return -tamsayı;
    }
    static int İkiTamsayıyıTopla(int tamsayı1,int tamsayı2)
    {
        return tamsayı1+tamsayı2;
    }
    static void Main(string[] args)
    {
        int sayı = KonsoldanBirTamsayıOku();
        TamsayıyıYaz(sayı);
        sayı=İşaretiniDeğiştir(sayı);
        TamsayıyıYaz(sayı);
        int toplam = İkiTamsayıyıTopla(sayı, 100);
        TamsayıyıYaz(toplam);
    }
}
```

Goto Talimatı

Tanımlı bir etikete program akışını yönlendiren **goto** talimatıdır. *Sayaç Kontrollü Döngüler* başlığında örneği verilmiştir.